RandAO

Fast | Fully Decentralized | Verifiable Randomness | Token Utility



January 26, 2025

CONTRIBUTORS

Allan B. Pedin IV
KENNY SWAYZEE

Alex Posey
CAITLYN LANE

Tyler Warburton
ETHAN GOLD

ABSTRACT

We present RandAO, a novel decentralized random number generation protocol that achieves unprecedented robustness and liveness through an ”any honest” guarantee - ensuring randomness integrity as long as at least one participant remains honest. By integrating Verifiable Delay Functions (VDFs) with a commit-reveal scheme, RandAO prevents denial of service attacks and manipulation attempts while maintaining unpredictable output generation. The protocol’s economic security scales with total staked collateral, requiring complete provider set collusion for successful attacks, making it suitable for high-stakes applications like universal lotteries. We introduce the $RNG token economy that creates a sustainable marketplace for randomness generation, where providers compete on service quality and earn rewards through both direct service payments and staking mechanisms. This dual incentive structure, combined with the protocol’s robust security model, establishes a foundation for trustless on-chain randomness.

CONTENTS

1 Introduction **1**

1.1 Motivation 1

1.2 Our Contribution 1

2 Background **2**

2.1 Verifiable Delay Functions 2

2.2 EVM Chains 2

2.3 AO 2

2.4 Pietrzak's VDF 3

3 Related Work **5**

3.1 Commit Reveal Scheme 5

3.1.1 Naive Implementation Vulnerability 6

3.2 Oracle Based VRF 7

4 RandAO Protocol **8**

4.1 Protocol 8

4.2 Timing 10

4.2.1 Case: Standard 10

4.2.2 Case: Provider Fails to Post Proof 11

4.3 Economic Security 12

5 RNG Economy **13**

5.1 Token Utility 13
5.2 Supply and Demand 13
5.3 Market Dynamics 13
5.4 Staking Mechanism 14

6 Applications **15**

6.1 Gaming 15
6.2 NFTs 15
6.3 Proof of Stake Consensus 15
6.4 Artificial Intelligence 16

7 Future Developments **17**

7.1 Subscription RNG Service Layer 17
7.2 Honey Pot Mechanism 17

8 Outro **19**

INTRODUCTION

1.1. MOTIVATION

Random values have been a fundamental building block of Web2 applications, powering everything from gaming mechanics to cryptographic protocols. As we transition to Web3, we must maintain these essential capabilities while upholding our core principles of decentralization and security. However, blockchain's inherently deterministic and public nature presents a unique challenge in preserving the unpredictability of random values.

This fundamental limitation creates what we term the "**Onchain Randomness Problem**": as soon as entropy becomes available on the blockchain, all outcomes dependent on that entropy become predictable by any observer. Traditional attempts to generate randomness within blockchain systems have led to various compromises in security, integrity, or decentralization, undermining the very principles Web3 aims to uphold.

1.2. OUR CONTRIBUTION

We present a novel protocol for decentralized randomness generation that operates through an offchain open participation provider network. Our key contribution is a protocol design that enables secure onchain randomness aggregation while maintaining strong security properties:

- **Any-Honest Security**: The protocol remains secure as long as at least one participant in the network is honest.
- **Information Theoretic Security**: The protocol design ensures that randomness providers receive no additional information that could compromise the system's integrity or predictability.
- **Open Participation**: Any participant can join the provider network, fostering true decentralization while maintaining security through cryptographic guarantees rather than trust.

Through this design, RandAO achieves both the security properties of decentralized systems and the practical requirements for reliable randomness generation in blockchain environments.

BACKGROUND

2.1. VERIFIABLE DELAY FUNCTIONS

A Verifiable Delay Function (VDF) is a cryptographic primitive that requires a specified number of sequential steps to evaluate (and hence a theoretical amount of time), yet produces a unique output that can be efficiently verified [1].

Two key properties make VDFs particularly useful:

- **Sequential Computation**: The evaluation must be inherently sequential, meaning even with parallel processing, the computation cannot be significantly accelerated.
- **Efficient Verification**: While evaluation is intentionally slow, verification of the result can be performed quickly, typically in logarithmic time relative to the evaluation time.

By their nature, VDFs require large amounts of compute steps both for their evaluation and verification. Prior to the introduction of AO, executing a VDF within a decentralized smart contract platform would have been impractical.

2.2. EVM CHAINS

Ethereum revolutionized the decentralized world by introducing a first of its kind general purpose decentralized verifiable compute platform. This breakthrough unlocked the door for a variety of decentralized applications. However, like many layer-1 blockchains, it faces inherent scalability challenges. As noted in the original whitepaper, “Like Bitcoin, Ethereum suffers from the flaw that every transaction needs to be processed by every node in the network” [2].

2.3. AO

AO introduces a novel approach to decentralized computing, establishing a smart contract platform that achieves unprecedented scalability through its unique architecture [3].

Built on top of the Arweave network, AO distinguishes itself from traditional blockchain-based smart contract platforms through its actor-oriented design and message-passing paradigm.

Unlike conventional smart contract platforms that face inherent scalability limitations, AO enables arbitrarily scaling parallel computation where processes communicate through message passing rather than shared state. This breakthrough enables, for the first time, trustless parallel processing at scale, unlocking an entirely new class of decentralized applications previously impossible on traditional blockchain platforms.

The implications of this architecture are far-reaching: compute-intensive applications like Verifiable Delay Functions and Large Language Models can now be executed efficiently in a fully decentralized manner. This represents a fundamental shift in what's possible in the decentralized world.

2.4. PIETRZAK'S VDF

Pietrzak [4] presents an elegant yet simple implementation of a Verifiable Delay Function using the RSW timelock puzzle [5]. The construction is defined by:

- N: An RSA modulus
- x: Input value in $\mathbb{Z}_N^*$
- T: Number of squaring operations (determines delay length)
- n: Number of checkpoints (determines verification parallelization)
- y: Final output value

The core computation follows the formula:

$$y = x^{2^T} \bmod N$$
$$\mu_i = x^{2^{iT/(n+1)}} \bmod N \text{ for } i \in \{1, \ldots, n\}$$

These n checkpoints split the verification into $n+1$ equal segments, each requiring $T/(n+1)$ squaring operations that can be verified independently.

To illustrate with the simplest case, consider a single checkpoint ($n = 1$):

$$\mu = x^{2^{T/2}} \bmod N$$
$$y = x^{2^T} \bmod N$$

This splits the verification into two equal segments:

- Segment 1: $x \to \mu$ (first $T/2$ squarings)
- Segment 2: $\mu \to y$ (final $T/2$ squarings)

These segments can be verified in parallel, reducing verification time by approximately 50%. This pattern extends naturally to any number of checkpoints n, with each additional checkpoint further subdividing the verification process.

RELATED WORK

3.1. COMMIT REVEAL SCHEME

The commit reveal scheme is a decentralized protocol for generating verifiable random numbers. Let ε denote entropy and P represent a provider. The Provider Set is defined as:

$$\mathcal{P} = \{p_1, p_2, ..., p_n\} \tag{1}$$

where any participant can post collateral and run a provider node to join.

A commitment is defined as the hash of an entropy value:

$$c_i = \text{hash}(\varepsilon_i) \tag{2}$$

where c_i is the commitment from provider p_i. The protocol is executed through a secure smart contract that handles the aggregation of provider entropy values.

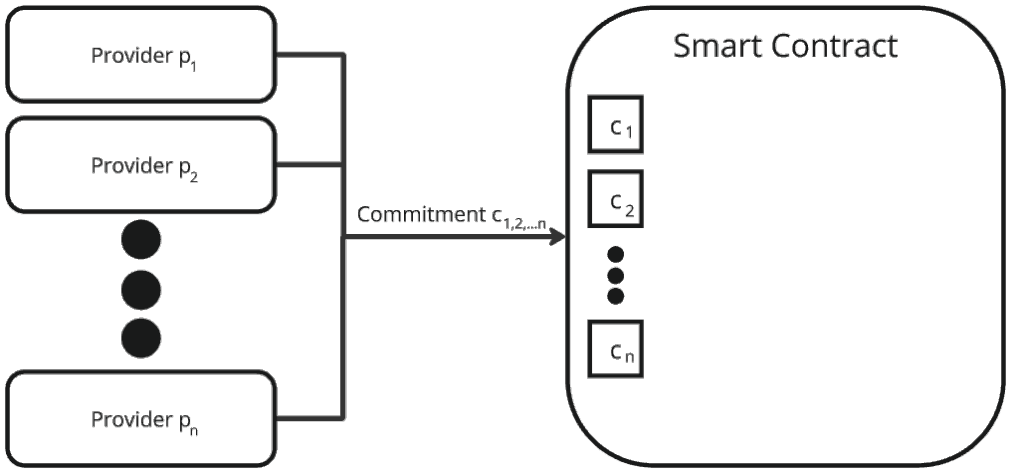


Figure 1: Commitment Stage: Providers simultaneously submit hash commitments of their entropy values

The protocol consists of the following key steps:

1. **Collateral**: Providers must post collateral to participate, ensuring economic security of the protocol
2. **Provider Commitment**: Each provider p_i commits to their entropy by submitting $c_i = \text{hash}(\varepsilon_i)$ to the smart contract

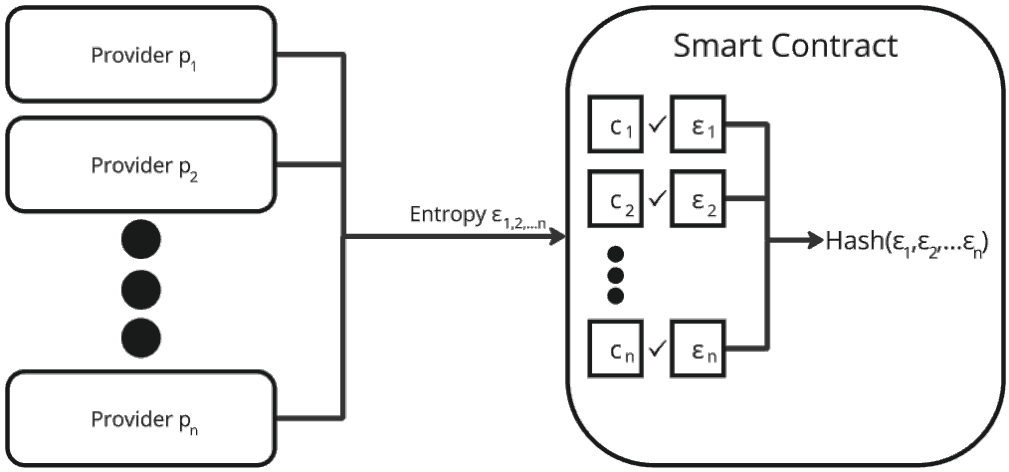


Figure 2: Reveal Stage: Providers reveal their original entropy values which are verified against their commitments. After verification, a verifiable entropy unknown to any individual provider is created using a hash function

3. **Provider Entropy**: After all commitments are received, providers reveal their original entropy values ε_i
4. **Verification**: The smart contract verifies each revealed entropy matches its commitment by checking $c_i = \text{hash}(\varepsilon_i)$
5. **Entropy Aggregation**: After all submitted entropies are verified, a verifiable entropy unknown to any of the individual providers is created using a method such as a hash function

3.1.1 Naive Implementation Vulnerability

While this naive commit-reveal scheme achieves decentralization, it suffers from a critical any-malicious denial of service vulnerability. Consider the following lottery scenario:

Suppose a lottery is being conducted with an insurmountable monetary reward. A single malicious provider can exploit the protocol through the following attack vector:

1. The malicious provider initially follows the protocol through the commitment step, submitting their hash commitment like other participants.
2. During the entropy revelation step, this provider (who may have purchased any number of lottery tickets) can wait until all other providers have revealed their entropy values.
3. At this moment, the malicious actor can calculate the final outcome of the lottery by combining:
    - All revealed entropy values from honest providers

- Their own private (yet unrevealed) entropy

4. With knowledge of the final outcome, the malicious actor can:
    - If favorable: reveal their entropy and allow the lottery to complete
    - If unfavorable: withhold their entropy, forcing a protocol restart and a new chance at a favorable outcome

This vulnerability demonstrates how a single malicious actor can repeatedly force redraws until achieving their desired outcome, effectively compromising the fairness of any high-stakes random number generation.

3.2. ORACLE BASED VRF

Chainlink's Verifiable Random Function (VRF) [6] is based on the VRF construction proposed by Goldberg et al. [7]. The Goldberg VRF creates a pseudorandom number that is verifiable by the public. However, as noted in their work, it "guarantees that β is indistinguishable from random by anyone who does not know the secret key SK." Conversely, this implies that anyone in possession of the secret key can predict the "random" output.

This reliance on secret key holder(s) creates a fundamental trust assumption. Chainlink's implementation depends entirely on the trustworthiness of Chainlink and their ability to secure their secret key from hackers and corruption, creating a single point of failure.

RANDAO PROTOCOL

4.1. PROTOCOL

The RandAO protocol builds upon the commit-reveal scheme discussed in Section 3.1, addressing its vulnerabilities through the integration of Verifiable Delay Functions (VDFs). The protocol consists of several key phases:

1. **Collateral Phase**: All providers $p_i \in \mathcal{P}$ must stake a predetermined amount of collateral to participate, ensuring economic security.
2. **VDF Commitment Phase**: Upon request for randomness, providers post VDF commitments to the smart contract. As shown in Figure 3, each provider must submit their commitment before proceeding to the next phase.

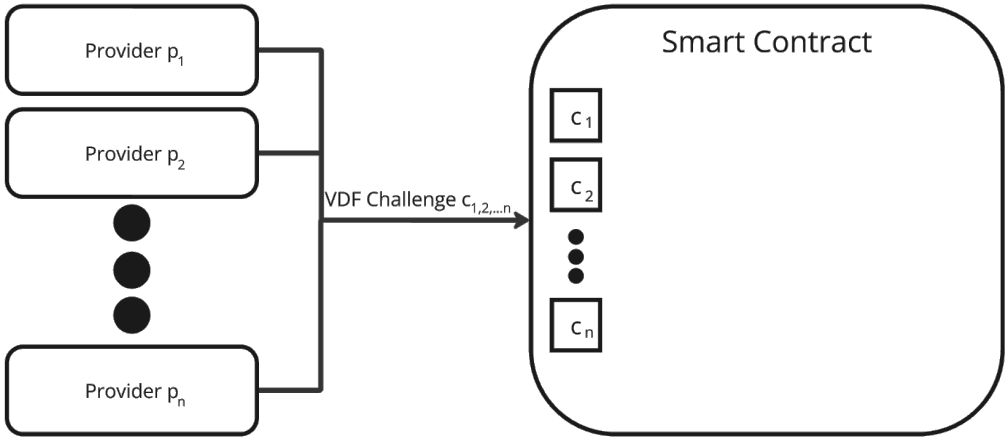


Figure 3: VDF Commitment Phase: Providers submit VDF commitments to the contract

Note: Once all commitments are posted, the final output entropy value is deterministically locked but requires time τ to calculate, where:

$$\tau = \frac{T}{s} \tag{3}$$

Here, T is the number of sequential squaring operations from Pietrzak's VDF construction, and s represents the number of modulo squaring operations per second achievable by the fastest available synchronous hardware [8]. For security, we implement a timeout where the protocol aborts if any provider fails to commit within time τ, resulting in penalties for non-compliant providers.

3. **VDF Proof and Verification Phase**: To expedite the final output calculation, providers submit VDF checkpoints that can be verified in parallel by the contract. This process differs based on provider behavior:

    - **Honest Case**: As illustrated in Figure 4, honest providers submit valid VDF proofs that are efficiently verified.

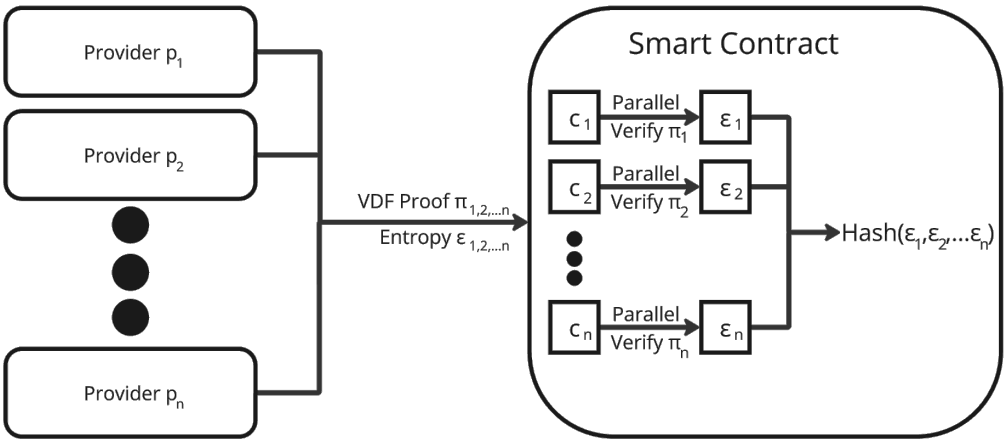


Figure 4: Verification of Honest Provider: Valid VDF proofs are efficiently verified

    - **Malicious Case**: Figure 5 demonstrates how the protocol handles malicious providers through verification and penalties.

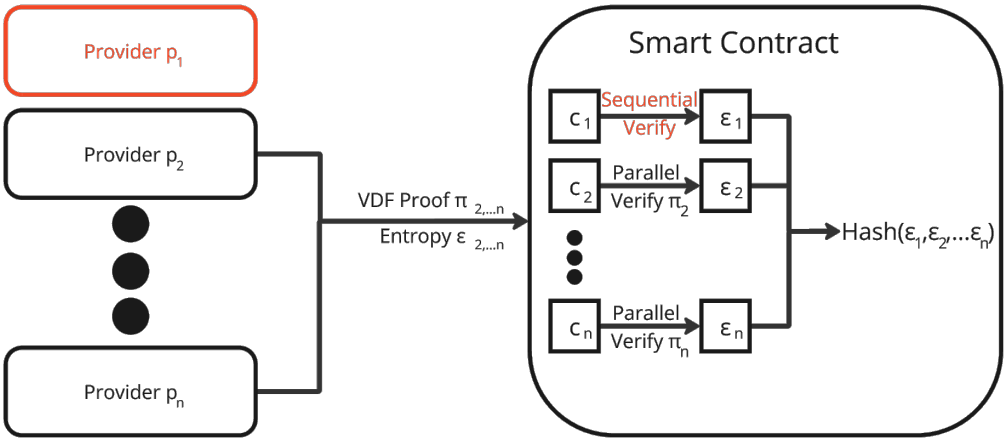


Figure 5: Malicious Providers during the proof stage are unable to abort the protocol, as their entropy value can be calculated sequentially from their VDF Challenge

4. **Entropy Aggregation**: After verification, all valid entropy values are aggregated using a cryptographic hash function to produce the final output entropy. This value remains unpredictable to any individual provider or participant, provided at least one provider acted honestly within the protocol.

This design addresses the denial of service vulnerability discussed in Section 3.1.1. Since all providers must submit VDF commitments and the final entropy value becomes deter-

ministically locked once all commitments are provided, a malicious attacker can, at worst, only delay the outcome by the predetermined time τ. This effectively solves the vulnerability where attackers could repeatedly force protocol restarts until achieving their desired outcome.

4.2. TIMING

The timing characteristics of the RandAO protocol, using Pietrzak's VDF construction (see Section 2.4), are primarily determined by three key parameters:

- T: The number of sequential squaring operations
- $\tau = T/s$: The actual time delay, derived from T and the fastest available hardware's squaring speed s [8]
- n: The number of checkpoints that enables parallel verification

These parameters establish the protocol's timing bounds, with T and the resulting τ determining the maximum delay an attacker can impose, and n optimizing the verification speed through parallelization. Below we examine how these parameters influence the protocol's timing in both standard operation and failure cases.

4.2.1 Case: Standard

The RandAO protocol's timeline in the standard case can be broken down into four distinct phases, as illustrated in Figure 6:

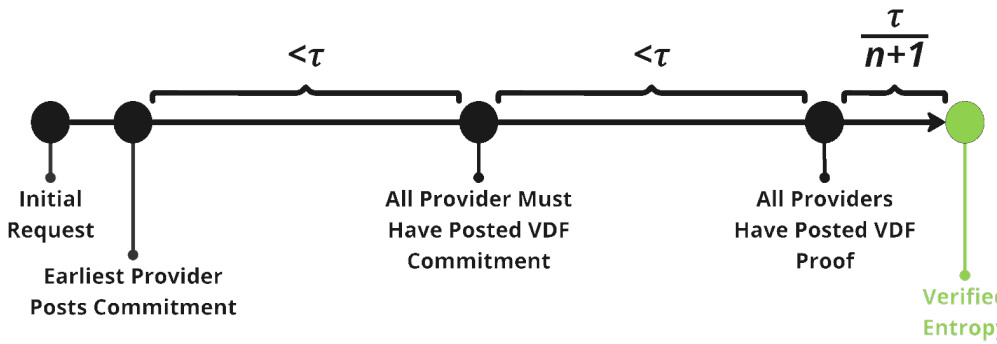


Figure 6: Timeline of the standard case scenario showing the four main phases of the protocol execution

1. **Initial Response**: The time between the initial random request and the first provider posting a VDF commitment. For competitive providers, this is expected to be near-instantaneous.

2. **Commitment Window**: The period between the first and final provider's VDF commitment posts. This window must complete within time τ to prevent providers from gaining additional information. The protocol will abort if this exceeds time τ. In practice, with competitive provider sets, this phase is expected to complete near-instantaneously.
3. **Proof Submission**: The duration between the final provider's VDF commitment and all providers submitting their VDF proofs. With competitive provider sets, this phase is also expected to complete near-instantaneously.
4. **Verification**: The time between all proofs being posted and the final verified entropy value being produced. Each proof segment takes time $\tau/(n+1)$ to verify, with all proofs and proof segments being verified in parallel:

$$t_{verify} = \frac{\tau}{n+1} \tag{4}$$

4.2.2 Case: Provider Fails to Post Proof

When a provider fails to post their VDF proof, the protocol timeline changes as illustrated in Figure 7:

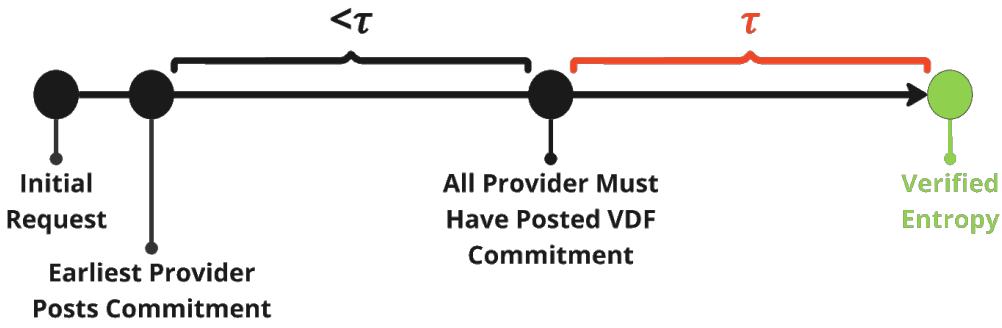


Figure 7: Timeline showing the protocol phases when a provider fails to post their proof

The first two phases remain identical to the standard case:

1. **Initial Response**: Near-instantaneous for competitive providers
2. **Commitment Window**: Must complete within time τ

However, the final phase differs significantly:

3. **Sequential Calculation**: The time required to sequentially calculate the non-compliant provider's entropy from their commitment, determined by τ. This represents the worst-case scenario where the protocol must complete the VDF computation without the provider's proof.

4.3. ECONOMIC SECURITY

RandAO's economic security model scales arbitrarily with the total staked collateral provided by the active provider set. A key strength of the protocol lies in its any-honest security guarantee: for any successful attack or collusion attempt to succeed, it must encompass every single member of the provider set. This creates a remarkably robust security model where a single honest citizen can uphold the integrity of an entire population - demonstrating that one individual's commitment to truth can outweigh any scale of corruption.

The security of the RandAO protocol scales with the relationship:

$$\mathcal{C} > \mathcal{R} \tag{5}$$

where $\mathcal{C}$ represents the total collateral at risk for a given provider set. As long as the potential rewards $\mathcal{R}$ remain below this threshold, the economic security of the protocol remains intact, as successful manipulation would require complete collusion of all providers - a requirement that scales in difficulty as the provider set grows. Theoretically this protocol can remain secure against collusion for arbitrarily large rewards like universal lotteries, given a sufficiently collateralized provider set.

RNG ECONOMY

5.1. TOKEN UTILITY

The $RNG token serves as the native utility token of the RandAO ecosystem, designed to capture the value of decentralized random number generation. This utility is enforced through protocol design: all randomness requests must be paid for in $RNG tokens, establishing a foundational and continuous demand for the token.

5.2. SUPPLY AND DEMAND

The RNG economy is driven by two primary mechanisms:

- **Demand Side**: Users requiring verifiable randomness must acquire and spend $RNG tokens to access the service. This creates natural market demand that scales with the protocol's adoption across various use cases such as gaming, NFTs, and blockchain consensus.
- **Supply Side**: $RNG tokens flow to providers through two channels:
  - Direct service payments from users requesting randomness
  - Staking rewards for maintaining collateral in the network

5.3. MARKET DYNAMICS

When randomness is requested, users can select any subset of providers within the open network. This flexibility creates a competitive marketplace where:

- Providers compete on service quality and reliability
- Market forces determine efficient pricing for randomness generation
- $RNG tokens are distributed proportionally to participating providers

5.4. STAKING MECHANISM

The protocol implements a robust staking mechanism:

- **Collateral Requirement**: Providers must stake wAR to participate
- **Security**: Staked assets ensure economic security against malicious behavior
- **Rewards**: Providers earn $RNG tokens as staking rewards
- **Alignment**: The dual reward structure (service fees + staking rewards) creates long-term alignment between providers and the protocol

APPLICATIONS

6.1. GAMING

Multiplayer video games extensively use random mechanics that directly impact player experiences and in-game economies. From critical hit calculations to loot drops and loot-box mechanics, players invest significant time and resources based on these probabilities. The fairness of these systems directly affects player satisfaction and game balance.

This need for verifiable randomness extends naturally into gambling applications, where every slot machine pull, card shuffle, and dice roll must be demonstrably random and tamper-proof to maintain player trust and regulatory compliance. The stakes are often significant, making manipulation attempts a serious concern.

6.2. NFTS

Non-Fungible Tokens (NFTs), such as those from the Bored Ape Yacht Club (BAYC) [9], are typically generated by combining various image layers to create unique digital collectibles. Due to the challenges associated with generating randomness directly on-chain—a situation often referred to as the ”On-Chain Randomness Problem” —these NFTs are frequently created off-chain by developers and then uploaded to the blockchain. This process can be susceptible to manipulation. Implementing true on-chain randomness would enable provably fair and transparent NFT minting processes, ensuring each token’s uniqueness and integrity.

6.3. PROOF OF STAKE CONSENSUS

Randomness is extensively utilized in Proof of Stake (PoS) consensus mechanisms. For instance, Solana employs random selection to determine the next leader in its consensus process [10]. This method ensures a fair and unpredictable rotation among validators.

6.4. ARTIFICIAL INTELLIGENCE

Random values are essential in neural networks such as large language models (LLMs) like ChatGPT because they help the system learn. One of the most important uses of randomness is weight initialization, where the initial values of connections (weights) are set unpredictably. If weights started from known values, an attacker or even the system itself could predict learning paths, reducing diversity and making the network less effective. Instead, true random initialization ensures that different neurons learn different features, making the network more powerful and capable of recognizing complex patterns. In machine learning, unpredictability is key to avoiding bias and ensuring robust, secure models [11].

FUTURE DEVELOPMENTS

This section explores enhancements to the RandAO protocol that, while not currently under development(yet), would provide significant additional value to RandAO.

7.1. SUBSCRIPTION RNG SERVICE LAYER

A subscription-based service layer would extend the protocol's utility for applications requiring frequent random values. This enhancement would introduce several key benefits:

- **Message Optimization**: By establishing long-term relationships between providers and applications, the protocol can reduce the number of inter-process messages required to obtain random values. Instead of initiating a new request flow for each value, applications could receive streams random values, significantly reducing latency and network overhead.
- **Negotiated Pricing**: Long-term service agreements would enable negotiated pricing models, offering cost predictability for applications with consistent randomness needs. This could include:
  - Volume-based discounts for high-frequency users
  - Fixed-rate packages for predictable budgeting

This enhancement would particularly benefit applications in gaming, DeFi, and other sectors requiring high-frequency random value generation, while maintaining the protocol's core security properties and decentralized nature.

7.2. HONEY POT MECHANISM

A honey pot mechanism could be implemented to strengthen the protocol's security by incentivizing the entrapment of maliciously colluding providers. This mechanism would work as follows:

- **Entrapment Opportunity**: Any party could attempt to prove that a provider set is colluding by demonstrating their ability to predict the entropy output before it is revealed.
- **Proof of Collusion**: The proof would involve submitting a predicted entropy value before the protocol's completion. If the prediction matches the final output, it serves as cryptographic evidence that the providers leaked information or coordinated their inputs.
- **Economic Penalties**: Upon successful proof of collusion:
  - The colluding providers' staked collateral would be slashed
  - A significant portion of the slashed funds would be awarded to the party that exposed the collusion
  - Remaining funds could be burned or redistributed to honest providers
- **Preventive Effect**: The mere existence of this mechanism would serve as a powerful deterrent against provider collusion, as any participant in a collusion scheme could potentially turn whistleblower for the reward.

This enhancement would add an additional layer of security to the protocol by creating a financial incentive for monitoring and reporting malicious behavior, effectively making collusion attempts more risky and less likely to occur.

OUTRO

This whitepaper has outlined the "**Onchain Randomness Problem**" and demonstrated how current solutions have made compromises in decentralization and security, leading to unsustainable centralization and denial of service attacks.

We introduced the novel RandAO protocol, which solves the onchain randomness problem through an any-honest open access randomness provider network. The protocol's innovative design ensures that as long as a single provider remains honest, the system maintains its security and integrity.

The $RNG token serves as the protocol's utility token for purchasing random values, creating a supply and demand driven economy to serve the outlined applications. This economic model aligns incentives between providers and users while ensuring the protocol's long-term sustainability.

With the RandAO protocol in place, fully decentralized onchain randomness will be readily available for all applications, enabling a new generation of secure and trustless Web3 services.

REFERENCES

[1] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch, "Verifiable delay functions," in *Advances in Cryptology – CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2018, p. 757–788. [Online]. Available: https://doi.org/10.1007/978-3-319-96884-1_25

[2] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," Ethereum, Whitepaper, 2014. [Online]. Available: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf

[3] S. Williams, I. Morozov, T. Wilson, T. Hall, V. Juliano, and A. Navarro, "The AO protocol: A decentralized open-access supercomputer," Draft, September 2024. [Online]. Available: https://vpydsw3vqn6vxnhyjdbz2vl4as3htw2gxk6qamufjk5h4biipgua.arweave.net/q_A5W3WDfVu0-EjDnVV8BLZ520a6vQAyhUq6fgUIeag

[4] K. Pietrzak, "Simple Verifiable Delay Functions," in *10th Innovations in Theoretical Computer Science Conference (ITCS 2019)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), A. Blum, Ed., vol. 124. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 60:1–60:15. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITCS.2019.60

[5] R. L. Rivest, A. Shamir, and D. A. Wagner, "Time-lock puzzles and timed-release crypto," USA, Tech. Rep., 1996.

[6] Chainlink Labs, "Verifiable random function (vrf) - explained," 2023, accessed: 2025-01-12. [Online]. Available: https://chain.link/education-hub/verifiable-random-function-vrf

[7] D. Papadopoulos, D. Wessels, S. Huque, M. Naor, J. Včelák, L. Reyzin, and S. Goldberg, "Making NSEC5 practical for DNSSEC," Cryptology ePrint Archive, Paper 2017/099, 2017. [Online]. Available: https://eprint.iacr.org/2017/099

[8] D. Zhu, Y. Song, J. Tian, Z. Wang, and H. Yu, "An efficient accelerator of the squaring for the verifiable delay function over a class group," in *2020 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 2020, pp. 137–140.

[9] "Bored ape yacht club," https://boredapeyachtclub.com/, accessed: 2025-01-21.

[10] A. Yakovenko, "Solana: A new architecture for a high performance blockchain," https://solana.com/, 2025, accessed: 2025-01-21.

[11] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, Y. W. Teh and M. Titterington, Eds., vol. 9. Chia Laguna Resort, Sardinia, Italy: PMLR, 13–15 May 2010, pp. 249–256. [Online]. Available: https://proceedings.mlr.press/v9/glorot10a.html